

[Autonomous R&D 2026 Day 1]

파이썬 프로그래밍

연세대학교 산업공학과

윤현수 교수

hs.yoon@yonsei.ac.kr

1. 교수소개

2. 파이썬 기초 문법

(변수, 자료형, 조건문, 반복문, 딕셔너리, 함수, 클래스)

3. NumPy + Maze

(배열 연산, 인덱싱, 조건 필터링, 미로 탐색 알고리즘)

4. Pandas + EDA + 전처리

(DataFrame, 필터링, groupby, merge, 결측치, 이상치, 인코딩)

5. 제조 데이터 Feature Engineering

(Route, Sensor, Wafer)

교수소개



윤현수 교수
연세대학교 산업공학과

- 現 연세대학교 산업공학과 학과장
- 現 SK 하이닉스 VPP 자문교수
- 現 강남 세브란스 의생명융합센터 겸임교수
- 現 대한산업공학회 이사
- 現 한국데이터마이닝학회 이사
- 現 Mayo Clinic Research Professional
- 前 뉴욕주립대학교 산업공학과 조교수
- 前 IISE 학술이사

- 다수 산업 인공지능 산학과제 연구책임자 진행 경험 보유
– SK hynix, 삼성전자, 삼성엔지니어링, HD 현대 일렉트릭, 현대자동차, 삼성디스플레이, LG CNS, 아모레 퍼시픽, 세브란스 병원
- Amazon, Mayo Clinic, Columbia University, Texas A&M 등과 국제 공동 연구 수행
- 시뮬레이션-실제 데이터 Adaptation, 비전·센서 기반 이상 탐지, 도메인 적응, 혐의 설비 탐지, 다중 시계열 데이터 예측·분류 등 전력 설비 운영 최적화에 필수적인 AI 기술 연구
- 최근 5년간 SCI 논문 34편(33편 JCR Q1, 1편 Q2) 게재, Google Scholar 인용 1364회, h-index 16을 기록, INFORMS 및 IISE 최우수 논문상 3회 수상
- ICLR, CVPR, ACL, EAACL 등 AI Conference 논문 게재

KFAI



지식융합인공지능 연구실은 데이터와 더불어 다양한 유형의 정보 기술(도메인 지식, 물리화적인 원리, 모델 내에서 정보, 학습되는 특질 등) 활용하여 창의적인 인공지능 기술들을 개발하는데 특화된 연구를 진행하고 있습니다.



- 가. 현재 석사과정, 석박통합과정, 박사과정 등 총 20명의 연구원들이 ML/DL 분야 연구 진행
- 나. 외국 학술지 논문 게재 : 최근 5년간 SCI 논문 33건, 심사 중인 논문 18건
- 다. 인공지능 SOTA(최고성능모델): 3개 기술 (도메인적응, 이상탐지)
- 라. 데이터 분석 전문 교육 / 평가 : 20건

산학협력 및 융합 연구

SK하이닉스-연세대학교



음성기반 치매 고위험군 탐지 멀티모달 RAG AI Assistant



DRAM 설계 최적화
EU Mask FCDU 예측



환경 변화 대응형 AI 비전 엔진



3D Spool Search 엔진개발



고압회전기 절연시스템 신뢰성



Vision AI Pattern Searcher
Vision AI Pattern Recognition



파우치 씰링 A D



시뮬레이션과 실시간 추론 기반 통합 설비 이상탐지 시스템 개발



혐의설비탐지

학부

- IIE 2107 데이터분석 2022-1, 2023-1, 2024-1
- IIE 3107 품질공학 2025-1, 2026-1
- ENG 3406 AI 공학응용 2024-2, 2025-1, 2026-1
- IIE 4123 딥러닝과 응용 2021-2, 2022-2, 2023-2, 2024-2, 2025-2

- 우수강의 교수상
2022, 2023, 2024,
2025
- 우수업적교수상(교육부문)
2022

대학원

- IIE 7721 딥러닝 이론 및 응용 I 2021-2, 2023-1, 2025-1, 2026-1
- IIE 7722 딥러닝 이론 및 응용 II 2022-2
- IIE 7725 산업 인공지능 특론 2024-2
- IIE 8560 확률모형 및 베이지안 추론 2025-2
- IDO 5002 통계분석이론 2025-1
- DSS 6008 딥러닝 2021-2, 2022-2, 2023-2, 2024-2, 2025-2
- DSS 6010 인공지능 2025-1, 2026-1
- DSS 6013 파이썬 프로그래밍 2021-2, 2022-2, 2023-2, 2024-2, 2025-2
- IDO 6005 산업 Agentic AI 특론 2026-2

Python 기초

(변수, 자료형, 조건문, 반복문, 딕셔너리, 함수, 클래스)

1. 변수와 자료형

변수(Variable)는 데이터를 담아 두는 공간

- 이름을 붙여 값을 저장하고 꺼내 쓸 수 있음
- Python의 주요 자료형은 아래와 같음

자료형	설명	제조 데이터 예시
str	문자열	lot_name = 'LOT001'
float	실수	yield_rate = 95.3
int	정수	wafer_count = 25
bool	참/거짓	is_pass = True

```
lot_name      = 'LOT001'    # str
yield_rate    = 95.3        # float
wafer_count   = 25          # int
is_pass       = True        # bool

print(type(lot_name))      # <class 'str'>
print(type(yield_rate))    # <class 'float'>

# 형변환
print(float('95.3') + 1)   # 96.3
print(str(25) + '장')       # '25장'
```

결과

```
<class 'str'>
<class 'float'>
96.3
25장
```

💡 자료형이 다르면 연산 오류. '87' + 1 → TypeError. float('87') + 1 처럼 형변환 필요

2. 조건문 (if)

if문은 조건의 참·거짓 여부에 따라 실행할 코드 블록을 선택하는 명령문임

- 조건이 여러 개일 경우 elif를 사용하며, 어느 조건도 만족하지 않으면 else 블록이 실행됨.

```
# if - elif - else
yield_rate = 75.0

if yield_rate >= 90:
    grade = 'A'
elif yield_rate >= 80:
    grade = 'B'
elif yield_rate >= 70:
    grade = 'C'
else:
    grade = 'F'

print(f'Yield: {yield_rate}% → Grade: {grade}')
```

결과

['불량', '불량', '정상', '정상', '정상']

결과

Yield: 75.0% → Grade: C

```
# 복합 조건 and / or / in
yield_rate = 88.0
equip_id = 'EQ1'
watch_list = ['EQ1', 'EQ3']

if yield_rate < 90 and equip_id in watch_list:
    print(f'{equip_id} 점검 필요')
```

결과

EQ1 점검 필요

```
# 불량 판정 패턴
Fault_prob = [0.7, 0.9, 0.1, 0.2, 0.4]
Result = []

for p in Fault_prob:
    if p >= 0.5:
        Result.append('불량')
    else:
        Result.append('정상')

print(Result)
```

3. 반복문 (for)

지정한 범위 내 원소를 순차적으로 꺼내가며 실행 문장을 반복 수행하는 명령문임

- 범위를 벗어나면 반복이 종료되므로 반복 횟수가 미리 결정됨.

기본 for

```
lots = ['LOT001', 'LOT002', 'LOT003']
for lot in lots:
    print(lot, '처리 중')
```

결과

```
LOT001 처리 중
LOT002 처리 중
LOT003 처리 중
```

range 사용

```
for i in range(1, 4):
    print(f'공정 {i} 단계')
```

결과

```
공정 1단계
공정 2단계
공정 3단계
```

핵심 패턴: for + if + append

```
# 수율 90% 미만 LOT 추출
yields = [95.3, 88.1, 91.5, 76.2, 99.0]
fail = []

for y in yields:
    if y < 90:
        fail.append(y)

print('불량 수율:', fail)

# List Comprehension - 같은 결과, 더 짧게
fail2 = [y for y in yields if y < 90]
print(fail2)
```

결과

```
불량 수율: [88.1, 76.2]
[88.1, 76.2]
```

→ fail = [] 빈 리스트를 먼저 만들고 / for로 순회하며 / if 조건 맞으면 append — AI 코드의 70% 이 패턴

4. 딕셔너리 (Dictionary)

딕셔너리는 데이터의 대응 관계를 나타낼 수 있는 자료형

- 키(Key)와 값(Value)이 한 쌍으로 구성됨. 중괄호 {}를 사용하며
- 각 요소는 Key:Value 형태로 이루어짐. 값은 중복될 수 있지만 키는 중복될 수 없음.

```
# 생성 & 조회
yield_data = {
    'EQ01': 92,
    'EQ02': 85,
    'EQ03': 88
}
print(yield_data['EQ01'])    # 92
print(yield_data.keys())
print(yield_data.values())

# 추가 / 수정 / 삭제
yield_data['EQ04'] = 76      # 추가
yield_data['EQ02'] = 90      # 수정
del yield_data['EQ03']       # 삭제
print(yield_data)
```

결과

```
92
dict_keys(['EQ01', 'EQ02', 'EQ03'])
dict_values([92, 85, 88])
{'EQ01': 92, 'EQ02': 90, 'EQ04': 76}
```

함수	의미
keys()	Key 반환
values()	Value 반환
items()	Key-Value 쌍 반환
get(k, 기본값)	Key 없을 때 기본값 반환

딕셔너리 집계 패턴

```
# 설비별 평균 수율
eq_sum, eq_cnt = {}, {}

for lot in lot_data:
    eq = lot['Equip_ID']
    eq_sum[eq] = eq_sum.get(eq, 0) +
    lot['Yield']
    eq_cnt[eq] = eq_cnt.get(eq, 0) + 1

avg = {eq: eq_sum[eq]/eq_cnt[eq] for eq in
eq_sum}
```

→ get(key, 0): Key가 없으면 0 반환 — KeyError 방지

5. 함수(Function)

같은 목적에 맞게 반복 사용되는 코드를 def 키워드로 묶어 이름을 붙인 것임

- 매개변수(Parameter)를 입력 받아 결과를 반환(return)함.

```
def judge_yield(lot_name, yield_rate, threshold=90):  
    if yield_rate >= threshold:  
        result = '정상'  
    else:  
        result = '불량'  
    return f'{lot_name}: {result} ({yield_rate}%)'  
  
print(judge_yield('LOT001', 87))  
print(judge_yield('LOT002', 92))
```

결과

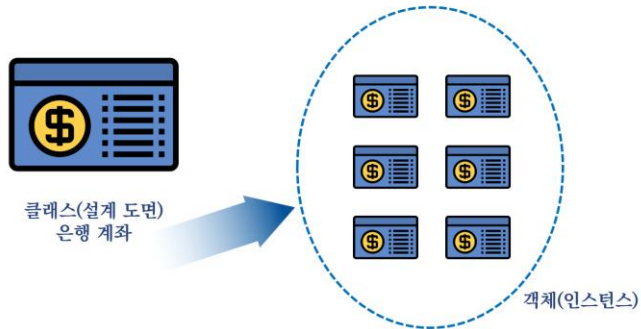
```
LOT001: 불량 (87%)  
LOT002: 정상 (92%)
```

Day 2에서 보게 될 Agent 패턴도 이 구조입니다: class QualityAgent: def run(self): ...

6. 클래스(Class)

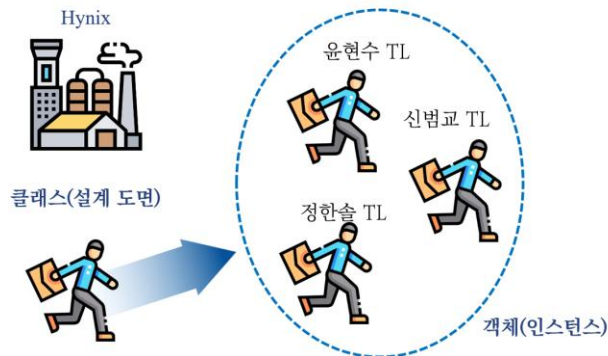
데이터(속성)와 그 데이터를 다루는 동작(메서드)을 하나로 묶어 정의하는 방식임.

- class로 설계도를 만들면 같은 구조의 객체를 여러 개 생성할 수 있음.



속성(Attribute): 잔액, 계좌의 주인, 이자율 등

행동(Method): 출금, 입금, 계좌 폐쇄 등



속성(Attribute): 근무지, 성별, 성격

행동(Method): 출근한다, 일한다, 밥 먹는다, 퇴근한다

```
class Equipment:
    def __init__(self, equip_id):
        self.equip_id = equip_id
        self.history = []

    def run(self, lot_name):
        self.history.append(lot_name)
        print(f'[{self.equip_id}] {lot_name} 완료')

    def report(self):
        print(f'{self.equip_id}: {self.history}')

eq1 = Equipment('ETCH01')
eq1.run('LOT001')
eq1.run('LOT002')
eq1.report()
```

결과

```
[ETCH01] LOT001 완료
[ETCH01] LOT002 완료
ETCH01: ['LOT001', 'LOT002']
```

NumPy + Maze

(배열 연산, 인덱싱, 조건 필터링, 미로 탐색 알고리즘)

NumPy는 수치 데이터를 빠르게 처리하기 위한 라이브러리임

- Python 리스트와 달리 배열 전체에 한 번에 연산을 적용할 수 있으며, 센서 데이터·수율 배열 처리에 필수적으로 사용됨.

```
import numpy as np

yield_arr = np.array([95.3, 88.1, 91.5, 76.2, 99.0])

print(yield_arr.mean().round(2)) # 평균
print(yield_arr.std().round(2)) # 표준편차
print(yield_arr.max())          # 최대
print(yield_arr.min())          # 최소

# Boolean Indexing - 조건 필터링
good = yield_arr[yield_arr >= 90]
print(good)
```

결과

```
90.02
8.43
99.0
76.2
[95.3 91.5 99. ]
```

```
# 2D 배열 - 센서 데이터 (행:Lot, 열:센서)
sensor_matrix = np.array([
    [10.1, 20.3, 30.5],
    [11.2, 21.1, 29.8],
    [10.8, 20.9, 31.0]
])

print(sensor_matrix.shape) # (3, 3)
print(sensor_matrix[1, :]) # 1행 전체
print(sensor_matrix[:, 1]) # 1열 전체
# axis=0: 센서별, axis=1: Lot별
print(sensor_matrix.mean(axis=0)) # 센서별 평균
```

결과

```
(3, 3)
[11.2 21.1 29.8]
[20.3 21.1 20.9]
[10.7 20.77 30.43]
```

💡 `yield_arr[yield_arr >= 90]` → 이것이 Boolean Indexing. Pandas의 `df[df['y']>=90]` 과 같은 원리

지금까지 배운 for, if, 함수, 배열, 시각화를 하나의 문제에 통합해 사용하는 실습임.

- 이 실습을 이해하면 Cursor가 생성하는 알고리즘 코드를 읽고 수정할 수 있음.

Step 1

30×30 배열 생성
(0=길, 1=벽)

```
maze = np.zeros([30, 30])
```

Step 2

외벽 추가
(0행·29행·0열·29열=1)

```
maze[0,:]=1 /  
maze[:,29]=1
```

Step 3

랜덤 내부 벽 120개
인접 벽 ≤1일 때만 추가

```
for _ in range(120):  
    ...
```

Step 4

랜덤 워크
10000번 이동

```
moves =  
[ (x+dx,y+dy) ...]
```

Step 5

목적지까지 최단 경로
& 시각화

```
plt.imshow(maze,  
cmap='binary')
```

미로 생성 함수로 만들기 – 재사용 가능

```
def generate_maze(n=30, num_walls=120, seed=42):  
    np.random.seed(seed)  
    maze = np.zeros([n, n])  
    for i in range(n):  
        maze[i,0]=1; maze[i,n-1]=1  
        maze[0,i]=1; maze[n-1,i]=1  
    for _ in range(num_walls):  
        x = np.random.randint(1, n-1)  
        y = np.random.randint(1, n-1)  
        nb = maze[x-1,y]+maze[x+1,y]+maze[x,y-1]+maze[x,y+1]  
        if nb <= 1:  
            maze[x, y] = 1  
    return maze
```

사용 문법	적용 위치
np.zeros	미로 배열 초기화
for + range	외벽/내벽 반복
if + 조건	벽 배치 제한
def 함수	재사용 가능한 생성기
plt.imshow	배열 → 그림 출력

Pandas + EDA + 전처리

(DataFrame, 필터링, groupby, merge, 결측치, 이상치, 인코딩)

DataFrame은 행(row)과 열(column)로 이루어진 2차원 표 형태의 자료구조임.

- 엑셀 시트와 구조가 유사하며, 코드로 조작하므로 반복 작업을 자동화할 수 있음.

```
import pandas as pd

df = pd.read_csv('Preprocessing_Data.csv',
                 encoding='EUC-KR')

print(df.shape)           # (행 수, 열 수)
df.head(3)                # 상위 3행
df.info()                 # 컬럼, 타입, 결측치 수
df.describe()             # 수치형 기술통계

# 컬럼 선택
print(df['y'].head())
df[['x2', 'y']].head()

# loc / iloc
print(df.loc[0, 'y'])     # 라벨 기반
print(df.iloc[0, 2])      # 숫자 위치 기반
```

```
# 조건 필터링
high_y = df.loc[df['y'] >= 4.5]
print(f'전체: {len(df)} / 조건: {len(high_y)}')

# 복합 조건 &=and  |=or
target = df.loc[
    (df['y'] >= 4.5) & (df['x2'] == 'EQ1')
]
print(target.shape)
```

결과

전체: 200 / 조건: 62
(28, 5)

메서드	의미
head(n)	상위 n행 출력
info()	타입·결측치 확인
describe()	수치형 기술통계
loc[조건]	조건 필터링 (라벨)
iloc[행,열]	위치 기반 선택

Groupby & merge

groupby — 설비별 집계

```
# 설비 (x2) 별 y 평균
df.groupby('x2')['y'].mean().round(3)

# 여러 통계 한 번에
df.groupby('x2')['y'].agg(['mean', 'std', 'count']).round(3)
```

결과

```
x2
EQ1    4.512
EQ2    4.283
EQ3    4.701
```

merge — 테이블 결합

```
lot_df = pd.DataFrame({
    'LotName': ['LOT001', 'LOT002', 'LOT003', 'LOT004'],
    'Process': ['1700A', '1300A', '4100A', '1700A']})
yield_df = pd.DataFrame({
    'LotName': ['LOT001', 'LOT002', 'LOT003'],
    'Yield': [95.3, 88.1, 91.5]})

# left join - 왼쪽 기준, 없으면 NaN
merged = pd.merge(lot_df, yield_df,
                  on='LotName', how='left')
```

결과

```
LotName Process Yield
0  LOT001  1700A   95.3
1  LOT002  1300A   88.1
2  LOT003  4100A   91.5
3  LOT004  1700A    NaN ← 없으면 NaN
```

how=	의미
'left'	왼쪽 DataFrame 모든 행 유지
'inner'	양쪽 모두 존재하는 Key만
'outer'	양쪽 합집합, 없으면 NaN

월드컵으로 연습

```
matches = pd.DataFrame({
    'Home':
    ['Mexico', 'Korea', 'Czechia', 'Mexico'],
    'Away':
    ['S.Africa', 'Czechia', 'S.Africa', 'Korea'],
    'Home_Score': [2, 2, 1, 1],
    'Away_Score': [0, 1, 1, 0]
})
# groupby로 팀별 포인트 집계 → 순위표 생성
# → 노트북 Cell 17~19 참고
```

→ 제조 데이터도 결국 여러 테이블을 LotName 기준으로 merge하는 것이 핵심

전처리: 결측치 · 이상치 · 인코딩

실제 데이터에는 결측치(NaN), 이상치, 분석에 필요 없는 컬럼이 섞여 있는 경우가 많음

- 인공지능 모델에 입력하기 전 반드시 아래 순서로 처리해야 함.

```
EQ = pd.read_excel('EQ_Analysis_data.xlsx')
data = EQ.copy() # 원본 보호

# ① 불필요 컬럼 제거
data = data.drop(['D1', 'D2', 'D3', 'D4', 'D5'], axis=1)
data = data.drop(
    data.columns[data.nunique()==1], axis=1) # 고유값 1개 제거

# ② 결측치 처리
data.fillna(data.mean(numeric_only=True), inplace=True)
data.fillna('unknown', inplace=True)

# ③ 이상치 제거 (IQR)
Q1, Q3 = data['A3'].quantile([0.25, 0.75])
IQR = Q3 - Q1
data = data.loc[(data['A3'] >= Q1-1.5*IQR) &
                (data['A3'] <= Q3+1.5*IQR)]

# ④ One-Hot Encoding
data = pd.get_dummies(data, columns=['A4'])
```

copy()

원본 수정 방지. 항상 먼저 실행

fillna(mean)

수치형 결측치 → 해당 컬럼 평균으로 채움

IQR

$Q1 - 1.5 \times IQR \sim Q3 + 1.5 \times IQR$ 범위 바깥 = 이상치

get_dummies

문자열 → 0/1 컬럼 변환 (ML 모델 입력용)

→ **EQ.copy()**로 항상 원본을 보존하고 **data**를 수정

netflix DataFrame에서 장르(Genre)별로 평점(Rating)이 가장 높은 작품 1개씩 추출

```
import pandas as pd

netflix = pd.DataFrame({
    'Title': ['Dark', 'Stranger Things', 'Squid Game',
             'Teach You a Lesson', 'Money Heist', 'The Crown'],
    'Genre': ['Sci-Fi', 'Sci-Fi', 'Thriller',
             'Drama', 'Thriller', 'Drama'],
    'Rating': [8.8, 8.7, 9.2, 8.9, 8.3, 8.7]
})

# 방법 1: groupby + idxmax()
idx = netflix.groupby('Genre')['Rating'].idxmax()
print(netflix.loc[idx, ['Genre', 'Title', 'Rating']])

# 방법 2: sort_values + drop_duplicates
result = (netflix
          .sort_values('Rating', ascending=False)
          .drop_duplicates('Genre')
          .sort_values('Genre'))
print(result[['Genre', 'Title', 'Rating']])
```

예상 결과

Drama → Teach You a Lesson (8.9)
Sci-Fi → Dark (8.8)
Thriller → Squid Game (9.2)

핵심 메서드

idxmax()
→ 최댓값의 인덱스 반환

drop_duplicates('Genre')
→ Genre별 첫 번째 행만 유지
(sort 먼저 하면 최고 평점이 첫 행)

결과

	Genre	Title	Rating
4	Drama	Teach You a Lesson	8.9
0	Sci-Fi	Dark	8.8
2	Thriller	Squid Game	9.2

제조 데이터 Feature Engineering

(Route, Sensor, Wafer)

```
# Yield_Raw_Data.csv
LotName Process Equip_ID Y
LOT001 1700A EQ01 76
LOT001 1700A EQ02 76 ← Rework (같은 공정 2회)
LOT001 1300A EQ03 76
LOT002 1700A EQ01 92
```

Yield_Raw_Data

Lot이 어떤 공정 → 어떤 설비를 거쳤는지 이력

→ LOT001 이 1700A를 2 번 거침 → Rework

```
# Sensor_Data / LOT001.csv
Step Sensor_1 Sensor_2
0 101.2 198.4
1 102.5 201.1
2 99.8 197.3
```

Sensor_Data

공정 중 센서가 측정한 시계열 값 (Lot별 파일)

→ Step 수가 Lot마다 다름 → 요약통계로 압축

```
# ImageData / LOT001_W01.csv
0 0 1 0
0 0 0 1 ← 1 = 불량 다이
1 0 0 0
```

ImageData

Wafer 다이맵 (0=양품, 1=불량)

→ Yield = 1 - 불량/전체

최종 목표: 세 파일을 하나의 분석 테이블로 합치기 → final_dataset.csv (168행 × 27열)

같은 Lot이 동일 공정을 두 번 처리한 이력(Rework)이 섞여 있음.

- 그대로 분석하면 중복 데이터가 들어감.

현재 형태

LotName	Process	Equip_ID	Event_Time	
LOT001	1700A	E01	2024-01-15 08:30	← 1차 처리
LOT001	1700A	E02	2024-01-15 10:10	← Rework!
LOT001	1300A	F01	2024-01-15 14:20	
LOT002	1700A	E01	2024-01-16 09:00	

```
route = pd.read_csv('Yield_Raw_Data.csv')
route['Event_Time'] =
pd.to_datetime(route['Event_Time'])
route =
route.sort_values('Event_Time').reset_index(drop=True)

# Rework 확인
dup = route[route.duplicated(
    subset=['LotName', 'Process'], keep=False)]
print(f'Rework 포함 행: {len(dup)}')
```

```
# Rework 제거 - 마지막 이력만 유지
route_clean = route.drop_duplicates(
    subset=['LotName', 'Process'],
    keep='last',
    ignore_index=True)
print(f'{len(route)} → {len(route_clean)} 행')
```

sort_values('Event_Time')

시간순 정렬 → 마지막 처리가 뒤에 오도록

drop_duplicates(subset=...)

(Lot, 공정) 조합이 같은 행 중 하나만 남김

keep='last'

중복 중 마지막 행 유지 → Rework 후 최종 처리

ignore_index=True

제거 후 인덱스를 0부터 다시 정렬

Route Table & 센서 Feature

Part B | pivot_table — 한 Lot = 한 행

```
route_table = route_clean.pivot_table(  
    index=['LotName', 'Number', 'Y'],  
    columns='Process',  
    values='Equip_ID',  
    aggfunc='first'  
)  
.reset_index()  
route_table.columns.name = None
```

LotName	1700A	1300A	4100A
LOT001	E02	F01	G03
LOT002	E01	F02	G01

→ 여러 행에 분산된 이력이 한 Lot 한 행으로 압축됨

Part C | 센서 Feature — 시계열 → 고정 벡터

```
sensor_rows = []  
for lot_file in lot_files:  
    lot_name = lot_file.replace('.csv', '')  
    df = pd.read_csv(f'{sensor_dir}/{lot_file}')  
    stats =  
df.describe().loc[['mean', 'std', 'min', 'max']]  
    row = stats.unstack() # Sensor_1_mean,  
    Sensor_1_std ...  
    row['LotName'] = lot_name  
    sensor_rows.append(row)  
  
sensor_feats = pd.DataFrame(sensor_rows)
```

Part D | Wafer Yield 계산

```
wafer_dir = f'{DATA_DIR}/ImageData'
wafer_files = os.listdir(wafer_dir)
wafer_df = pd.DataFrame(wafer_files,
                        columns=['WaferName'])

for i in range(len(wafer_df)):
    dm = pd.read_csv(
        os.path.join(wafer_dir,
            wafer_df.loc[i, 'WaferName']),
        header=None)
    wafer_df.loc[i, 'Yield'] = 1 -
        dm.values.sum() / dm.size
```

```
# 다이맵 예시
0 0 1 0
0 0 0 1    ← 1=불량
1 0 0 0
```

```
Yield = 1 - 3/12 = 0.75 (75%)
```

Part E | 최종 Merge

```
final_df = pd.merge(wafer_df, route_table,
                    on='LotName', how='left')
final_df = pd.merge(final_df, sensor_feats,
                    on='LotName', how='left')
final_df.to_csv('final_dataset.csv',
                index=False)
print(final_df.shape) # (168, 27)
```

감사합니다

연세대학교 산업공학과

윤현수 교수

hs.yoon@yonsei.ac.kr